FINAL YEAR PROJECT: PPT CONTENT GENERATION (SLIDES 1 - 10)

Domain: Software Testing & Automation (Eclipse, Java, Selenium)

Slide 1: Title + Team

- **Project Title:** Automated Functional Testing Framework Using Eclipse IDE and Selenium WebDriver.
- **Team Members:** [Your Name & Register Number]
- **Supervisor / Guide:** [Internal Guide Name]
- **Department:** Computer Science and Engineering / IT

Slide 2: Problem Statement

- **Manual Overheads:** Manual execution of repetitive test scenarios requires significant time and human resources.
- **Human Error Proneness:** High probability of missing minor UI defects or validation glitches during manual regression runs.
- **Lack of Reusability:** Test cases rewritten or re-evaluated manually for every new build without historical tracking.

Slide 3: Objectives

- **Automation of Test Cases:** To convert manual functional test cases into automated reusable test scripts.
- **Execution Time Reduction:** To significantly reduce execution time per test cycle from hours to minutes.
- **Accurate Defect Identification:** To generate precise logs and automated screenshots whenever a system assertion fails.

Slide 4: Existing vs Proposed System

- **Existing System:** Manual validation via Excel tracking sheet. Slower execution speed. Hard to repeat daily.
- **Proposed System:** Scripted automation framework using Eclipse IDE. High execution speed. Fully repeatable with 100% precision.

Slide 5: Features

- **Data-Driven Testing:** Capability to load dynamic inputs automatically from an external CSV/Excel sheet.
- **Real-time Assertion Checks:** Validates Expected Results vs Actual Results instantly during runtime.
- **Robust Error Handling:** Exception catching mechanisms to prevent script crashes on dynamic webpage changes.

Slide 6: Architecture Diagram Outline

- **Input Layer:** Functional Test Case Matrix (CSV/Excel Sheet).
- **Processing Engine:** Eclipse IDE with Java Development Kit (JDK) & Automation Scripts.
- **Drivers & Interface:** Selenium WebDriver managing browser instances (Chrome/Firefox).
- **Output Layer:** Result Logs, Console Output (JUnit/TestNG Pass-Fail reports), and Defect Screenshots.

Slide 7: Database / ER Diagram Model

- **Test Management Storage:** Abstract relational layout to log historical runs.
- **Tables Profile:** 1. *Test_Run* (Run_ID, Timestamp, Operator), 2. *Test_Cases* (Case_ID, Description, Category), 3. *Execution_Logs* (Log_ID, Status [Pass/Fail], Duration).

Slide 8: Matching & Assertion Algorithm

- **Step 1:** Initialize WebDriver and load configurations from the test script.
- **Step 2:** Read specific Test Input values line-by-line from the source CSV file.
- **Step 3:** Execute DOM interaction (Locating elements via ID, XPath, or Name).
- **Step 4:** Trigger Assertions: Compare *Expected Result* with *Actual UI Text*.
- **Step 5:** Log Result as PASS if matching, else catch exception, capture screenshot, and mark as FAIL.

Slide 9: Screenshots Checklist

- **Eclipse Code Workspace:** Clean capture of your Java Test Classes and project directories.
- **Execution Console:** Screenshot of the runtime logs indicating successful script execution.

- **Validation Reports:** JUnit or TestNG graphical color bar showing passed metrics.

Slide 10: Testing Methodologies

- **Functional Testing:** Verification of system behaviors against defined functional requirements.
- **Smoke & Sanity Runs:** Initial checks to ensure core build modules are fully operational.
- **Regression Testing:** Automated execution of existing test cases to confirm new updates didn't break old features.